

P0597

To: EWG

From: Daveed Vandevoorde (daveed@edg.com)

Date: 2017-02-02

`std::constexpr_vector<T>`

As `constexpr` functions become increasingly used for non-trivial computations, the lack of any form of dynamic allocation becomes increasingly limiting. Permitting the use of an `::operator new` in constant-expressions is interesting and probably not impossible, but it brings many specification subtleties and implementation difficulties. Rather than attempting that ambitious project, I'm proposing a new resizable container type `std::constexpr_vector<T>`, that is a literal type "by fiat". Objects of such a type can only be created in constant-expression evaluation contexts and only for types `T` that are literal types. Its interface is similar to that of `std::vector<T>` (without any mention of allocators), but invoking a mutating operation on a `constexpr_vector` is only permitted if the vector's lifetime begins and ends during the current constant-expression evaluation.

Examples

The following is valid:

```
constexpr constexpr_vector<int> x; // Okay.  
constexpr constexpr_vector<int> y{ 1, 2, 3 }; // Okay.
```

but the following is not:

```
const constexpr_vector<int> xe; // Invalid: not constexpr
```

because the latter attempts to create a `constexpr_vector` in a non-constexpr context. We'll discuss below what "invalid" should mean.

In a `constexpr` function, it is possible to mutate a `constexpr_vector` :

```
using Ints = std::constexpr_vector<int>;  
constexpr auto series(int n)->Ints {  
    Ints r{};  
    for (int k; k<n; ++k) r.push_back(k);  
    return r;  
}
```

Such a function can only be invoked in a constant-expression context:

```
constexpr Ints digits = series(10); // Okay.  
const int twenty = *(series(21).end()-1); // Invalid.
```

Literal class types can embed a `std::constexpr_vector` :

```
template<typename T> struct hash_table {  
    ...  
private:  
    std::constexpr_vector<T> table;  
};
```

but the resulting type is subject to the same constexpr context requirements:

```
const hash_table my_map{ ... }; // Invalid.
```

The contents of a `constexpr` `constexpr_vector` can be accessed as constant expressions. For example:

```
constexpr Ints digits = series(100);
std::array<int, digits.size()>
    digits_array(digits.begin(), digits.end());
```

What does `invalid` mean

Consider this example:

```
constexpr void f(int n) {
    if (n > 0) {
        constexpr_vector<int> x{};
    }
}
extern int N;
int main() {
    f(N);
}
```

A priori, there is nothing invalid about the definition of `f`: `constexpr_vector` objects can potentially be created in `constexpr` functions. The call to `f` in `main` is not a constant-expression, however, which makes the potential creation of a `constexpr_vector` during that call suspect. On the other hand, it is very possible that `N` will never be positive, and that therefore no `constexpr_vector` is ever created.

How, then, should we make the case where `N` is strictly positive invalid?

More generally, how much effort should an implementation spend on diagnosing the cases that are obviously wrong?

I propose the following:

- A construction of a `constexpr_vector` object or subobject that is neither a constant-expression nor occurring during `constexpr` function call is ill-formed and must be diagnosed.
- A construction of a `constexpr_vector` object or subobject during a `constexpr` function call that is not part of a constant-expression evaluation calls `std::terminate`.

There are alternatives:

- Instead of `std::terminate`, we could invoke undefined behavior. This is easier.
- We could make `constexpr_vector` a type that works in non-constant-expression contexts too. I ran into specification and implementation difficulties exploring this possibility and am therefore not proposing it. (In particular, it essentially requires that `constexpr_vector` have a nontrivial destructor which makes the `constexpr` evaluation model more complex.)

Synopsis

```
namespace std {
    template<typename T> struct constexpr_vector {
        using value_type = T;
        using pointer = T*;
        using const_pointer = T const*;
        using reference = T&
        using const_reference = T const&
        using size_type = std::size_t;
        using difference_type = std::ptrdiff_t;
        using iterator = T*;
        using const_iterator = T const*;
        using reverse_iterator = std::reverse_iterator<T*>;
```

```

using const_reverse_iterator = std::reverse_iterator<T const*>;

constexpr constexpr_vector();
    constexpr explicit constexpr_vector(size_type);
    constexpr explicit constexpr_vector(size_type, T const&);
    template<typename InputIterator> constexpr
        constexpr_vector(InputIterator first,
                        InputIterator last);
constexpr constexpr_vector(constexpr_vector const&);
constexpr constexpr_vector(constexpr_vector&&);
constexpr constexpr_vector(initializer_list<T>);
// No user-provided destructor.

constexpr constexpr_vector& operator=(constexpr_vector const&);
constexpr constexpr_vector& operator=(constexpr_vector&&);
constexpr constexpr_vector& operator=(initializer_list<T>);
template<typename InputIterator> constexpr
    assign(InputIterator first, InputIterator last);
constexpr void assign(size_type, T const&);
constexpr void assign(initializer_list<T>);

constexpr iterator begin();
constexpr const_iterator begin() const;
constexpr iterator end();
constexpr const_iterator end() const;
constexpr reverse_iterator rbegin();
constexpr reverse_const_iterator rbegin() const;
constexpr reverse_iterator rend();
constexpr reverse_const_iterator rend() const;
constexpr const_iterator cbegin() const;
constexpr const_iterator cend() const;
constexpr reverse_const_iterator crbegin() const;
constexpr reverse_const_iterator crend() const;

constexpr bool empty() const;
constexpr size_type size() const;
constexpr size_type max_size() const;
constexpr size_type capacity() const;
constexpr void resize(size_type);
constexpr void resize(size_type, T const&);
constexpr void reserve(size_type);
constexpr void shrink_to_fit();

constexpr reference operator[](size_type);
constexpr const_reference operator[](size_type) const;
constexpr reference at(size_type);
constexpr const_reference at(size_type) const;
constexpr reference front();
constexpr const_reference front(size_type) const;
constexpr reference back();
constexpr const_reference back(size_type) const;

constexpr T* data();
constexpr T const* data() const;

template<typename ... Args> constexpr
    reference emplace_back(Args&&...);
constexpr void push_back(T const&);
constexpr void push_back(T&&);
constexpr void pop_back();
template<typename ... Args> constexpr
    iterator emplace(const_iterator, Args&&...);
constexpr iterator insert(const_iterator, T const&);
constexpr iterator insert(const_iterator, T&&);
constexpr iterator insert(const_iterator, size_type, T const&);
template<typename InputIterator> constexpr
    iterator insert(const_iterator, InputIterator, InputIterator);
constexpr iterator insert(const_iterator, initializer_list<T>);
constexpr iterator erase(const_iterator);
constexpr iterator erase(const_iterator, const_iterator);
constexpr void swap(vector&);

```

```
constexpr void clear();  
};  
}
```